	AVALIAÇÃO 3 - 2ª CHAMADA	Nota

Curso *Engenharia de Software*	**Campus** *Rio Verde*
Lógica para Engenharia de Software	
Nome do(a) acadêmico(a) EDUARDO VAZ MOREIRA BORGES	**Valor** 7,0
Professor *Sergio Souza Novak*	**Turma** *A*
Data da Avaliação	**Código** *ESW418*

ATENÇÃO: Somente serão passíveis de **REVISÃO** avaliações resolvidas à **TINTA**.

ORIENTAÇÕES PARA A RESOLUÇÃO

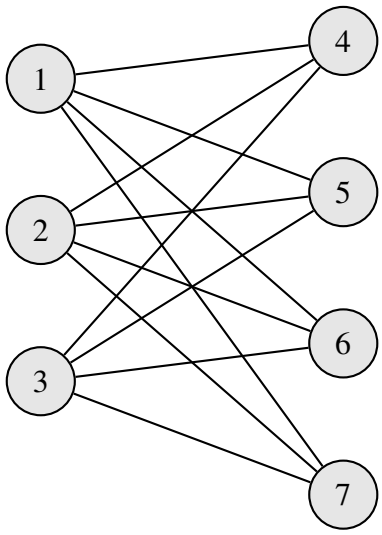
- A avaliação é individual, com consulta apenas ao material escrito a mão (à caneta);
- A interpretação de texto faz parte da avaliação;

QUESTÕES

1. **(ENADE) - Adaptada** No processo de pesquisa binária em um vetor ordenado, os números máximos de comparações necessárias para se determinar se um elemento faz parte de vetores com tamanhos 50, 220 e 300 são, respectivamente, iguais a

   (A) 5, 100 e 30.
   (B) 6, 8 e 9.
   (C) 8, 31 e 18.
   (D) 10, 100 e 30.
   (E) 25, 500 e 150.

2. Converta o grafo bipartido abaixo para o formato de Matriz de Adjacência, nos moldes do livro Cormen.



a) Preencha a matriz de adjacência abaixo.

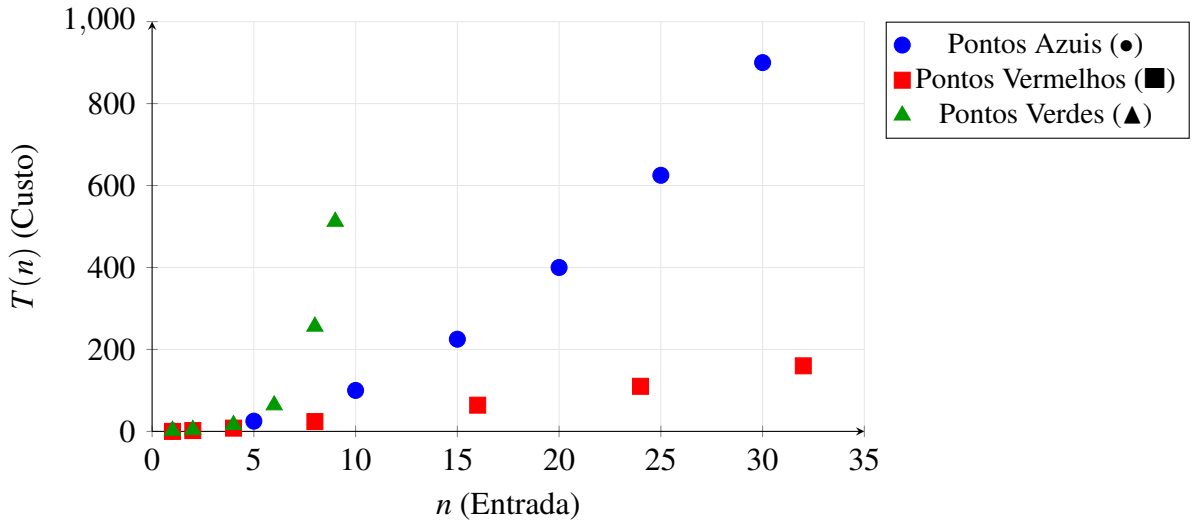
b) Explique, em suas palavras, porque um grafo bipartido é um tipo especial de grafo.

c) Qual é a vantagem da matriz de adjacência em relação à lista para este grafo?

Vértices	**1**	**2**	**3**	**4**	**5**	**6**	**7**
1							
2							
3							
4							
5							
6							
7							

Resposta:

3. Com base no comportamento das curvas no gráfico abaixo, assinale a alternativa que associa corretamente a cor/formato do ponto à sua respectiva complexidade:



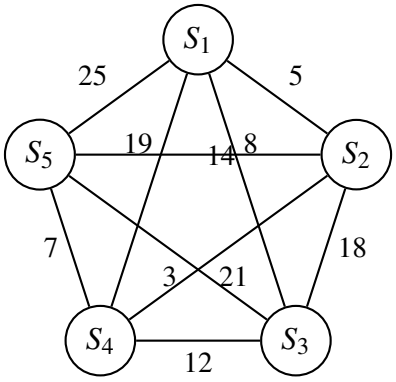
a) Azul: $O(n^2)$; Vermelho: $O(n \log n)$; Verde: $O(2^n)$.

b) Azul: $O(n^2)$; Vermelho: $O(2^n)$; Verde: $O(n \log n)$.

c) Azul: $O(2^n)$; Vermelho: $O(n \log n)$; Verde: $O(n^2)$.

d) Azul: $O(n \log n)$; Vermelho: $O(n^2)$; Verde: $O(2^n)$.

4. O algoritmo abaixo (Floyd-Warshall) calcula as distâncias mínimas entre todos os pares de vértices em um grafo (representado por uma matriz de adjacência W de pesos).



FLOYD-WARSHALL(W, n)

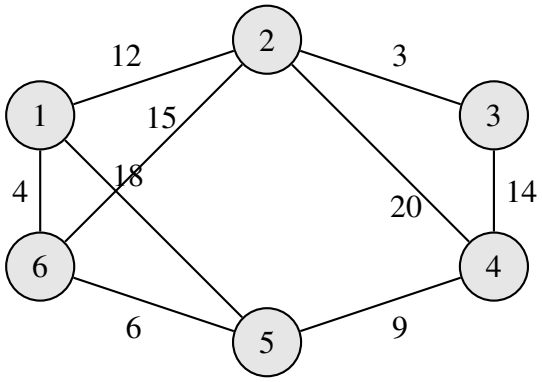
	instrução	**custo**	**vezes**
1	$D \leftarrow W$	_____	_____
2	**para** $k = 1$ **até** n:	_____	_____
3	**para** $i = 1$ **até** n:	_____	_____
4	**para** $j = 1$ **até** n:	_____	_____
5	**se** $D[i, j] > D[i,k] + D[k, j]$:	_____	_____
6	$D[i, j] \leftarrow D[i,k] + D[k, j]$	_____	_____
7	**retornar** D	_____	_____

a) Preencha as colunas **custo** e **vezes** para cada linha do algoritmo. (Considere o pior caso para a linha 6)

b) Determine a complexidade assintótica $T(n)$ do algoritmo no pior caso, expressando-a em notação Big-O.

Resposta:

5. Um novo condomínio industrial deseja conectar seus 6 galpões principais (identificados pelos números de 1 a 6) a uma central unificada de energia elétrica. O projeto exige que todos os galpões fiquem interligados (seja de forma direta ou indireta) na mesma rede, mas a construtora precisa realizar a obra com o menor orçamento possível. Os custos orçados (em milhares de reais) para o cabeamento subterrâneo entre cada par de galpões viável estão representados no projeto topológico abaixo.



a) Aplique o algoritmo de Kruskal para encontrar a Árvore Geradora Mínima (MST). Liste as arestas em ordem crescente de peso.

b) Para cada aresta analisada, indique se ela foi **adicionada** à MST ou **rejeitada** (por formar ciclo). Mostre a evolução dos conjuntos de vértices (simulando a estrutura Union-Find).

c) Desenhe a Árvore Geradora Mínima resultante.

d) Qual é o custo total mínimo para interligar os 6 galpões?

Resposta:

6. **Texto de Apoio:**

PEDROSO, Alexandre da Silva; CINTRA, Fausto Gonçalves. **Estudo analítico do desempenho de algoritmos de ordenação**. Revista EduFatec: educação, tecnologia e gestão, Franca, v. 2, n. 5, p. 73–88, ago./dez. 2022. Disponível em: Revista EduFatec – artigo em PDF. Acesso em: 25 jun. 2026.

O que é ordenar dados? Por que é tão importante ordenar dados em computação? Tome-se o caso de alguém que precise localizar o número de celular de um amigo na lista de contatos do seu celular, mas esses contatos não estão organizados sob nenhuma ordem. Caso a quantidade de contatos na lista seja elevada, essa busca pode se tornar uma atividade muito laboriosa e por vezes até mesmo infrutífera. Não por acaso, a lista de contatos dos celulares costuma estar organizada em ordem alfabética. Pode-se citar também a busca de informações sobre um cliente no banco de dados de uma empresa que possui milhares de cadastros. Se os dados se encontram ordenados, a busca pela informação se torna menos complexa. Por esses e outros motivos os algoritmos de ordenação se tornam um importante recurso utilizado por cientistas da computação.

Neste estudo, conforme antes anunciado, serão discutidas as características e a eficiência temporal dos algoritmos de ordenação *bubble sort*, *selection sort* e *quick sort*, tendo sido recolhidos dados a esse respeito por meio de pesquisa bibliográfica.

O algoritmo *bubble sort* (ordenação por flutuação) compara cada dois valores adjacentes, realizando a permuta entre eles se o primeiro valor for maior do que o segundo. O melhor caso ocorre quando o vetor a ser ordenado já está na ordem correta. Nesse caso, nenhuma troca será realizada e a estrutura de repetição será executada uma única vez, tendo sido realizadas $n-1$ comparações. Com isso diz-se que, no melhor caso, esse algoritmo possui complexidade $\Omega(n)$, utilizando a notação Ômega (Ω) para representar o tempo mínimo de execução. O pior caso

ocorre quando o vetor a ser ordenado está na ordem inversa à desejada. Logo, o número de comparações realizadas será $n(n-1) = n^2 - n$. Isso leva a concluir que, no pior caso, o tempo de execução do algoritmo é $O(n^2)$. O caso médio para esse algoritmo será $\frac{(n^2-n)+(n-1)}{2} = \frac{1}{2}(n^2-1)$ e, portanto, a complexidade do caso médio para o algoritmo será $O(n^2)$.

O algoritmo *quick sort*, primeiramente seleciona um valor do vetor, o qual é designado como pivô. Na primeira passada, divide o vetor em um subvetor à esquerda contendo apenas valores menores que o pivô, e outro subvetor à direita, que contém apenas valores maiores que o pivô. Em seguida, recursivamente, repete o processo em cada um dos subvetores até que todo o vetor esteja ordenado. Uma característica peculiar do *quick sort* é sua ineficiência para vetores já ordenados quando a escolha do pivô é inadequada. Nessa situação, as partições serão extremamente desiguais. Essa situação é desastrosa, pois o número de comparações passa a cerca de $\frac{n^2}{2}$, e o tamanho da pilha necessária para as chamadas recursivas é cerca de n. A melhor situação possível ocorre quando cada partição divide o vetor em duas partes iguais. Logo, $T(n) = 2T(\frac{n}{2}) + n - 1$. A solução para esta recorrência é: $T(n) = n\log n - n + 1$, o que define uma complexidade de melhor caso de $\Omega(n\log n)$. No caso médio, o número de comparações realizadas é $T(n) \approx 1,386n\log n - 0,846n$, o que significa que em média o tempo de execução do *quick sort* é $O(n\log n)$.

O algoritmo *selection sort* (ordenação por seleção) encontra primeiro o valor mínimo entre os dados de entrada, coloca esse valor na primeira posição, depois encontra o segundo valor mínimo colocando-o na segunda posição, e faz isso sucessivamente até que o vetor esteja ordenado. Para esse algoritmo, ao utilizarmos um vetor com n posições, o tempo de execução pode ser encontrado pela soma $T(n) = 1+2+3+\cdots+(n-1)$, que é uma progressão aritmética cuja soma resulta em $T(n) = \frac{1}{2}(n^2-n)$. Sendo assim, como ele sempre executa esse mesmo número de comparações independente da entrada, é possível concluir que a complexidade temporal do algoritmo *selection sort* é delimitada inferiormente por $\Omega(n^2)$.

Com base no texto de apoio, os três algoritmos de ordenação apresentam diferentes níveis de eficiência temporal dependendo da disposição inicial dos dados.

Assinale a alternativa que apresenta o algoritmo que tem o **melhor caso** mais eficiente:

a) Bubble sort

b) Quick sort

c) Selection sort

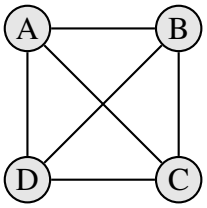
d) A eficiência dos três algoritmos no melhor caso é a mesma.

7. Tanto o algoritmo de Prim quanto o algoritmo de Kruskal são amplamente utilizados para encontrar a Árvore Geradora Mínima (MST) de um grafo não-direcionado e ponderado. Explique, de forma discursiva, qual é a principal diferença na estratégia de construção da MST entre os dois algoritmos e em quais cenários (relacionados à densidade do grafo) é mais vantajoso utilizar o Prim e em quais é mais recomendado o uso do Kruskal.

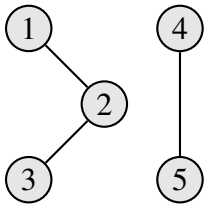
Resposta:

8. Analise os quatro grafos a seguir (I, II, III e IV):

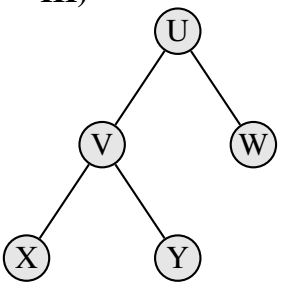
I)



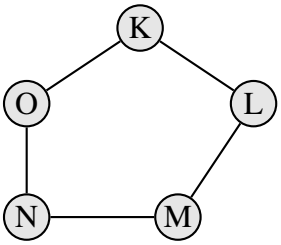
II)



III)



IV)



Sobre os grafos desenhados acima, assinale a alternativa **CORRETA**:

a) O Grafo I é uma árvore, pois não possui ciclos.
b) O Grafo III é classificado como completo, pois não possui ciclos.
c) O Grafo II é uma árvore, portanto não poderia possuir duas componentes conexas.
d) O Grafo IV possui árvore geradora.
e) O Grafo I possui quatro componentes conexas e é um grafo completo.

FORMULÁRIO DE APOIO

Consulta permitida durante a prova. Não destacar.

---

1. Teorema do Aperto de Mãos (*Handshaking Lemma*)

Em qualquer grafo $G = (V, E)$, a soma dos graus de todos os vértices é igual ao dobro do número de arestas:

$$\sum_{v \in V} \text{grau}(v) = 2 \cdot |E|$$

Consequência: Se todos os n vértices possuem o mesmo grau d (grafo regular), então:

$$|E| = \frac{n \cdot d}{2}$$

---

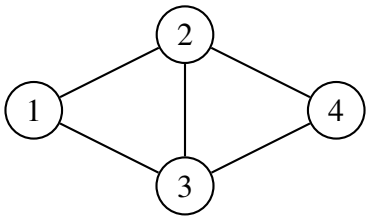
2. Algoritmo de Kruskal (Árvore Geradora Mínima — MST)

KRUSKAL(G, n)
1. Ordenar todas as arestas de G em ordem crescente de peso.
2. Inicializar cada vértice como um conjunto independente (Union-Find).
3. **Para** cada aresta (u, v) em ordem crescente de peso:
3.1. **Se** u e v pertencem a conjuntos **diferentes**:
Adicionar (u, v) à MST e unir os conjuntos de u e v.
3.2. **Senão**: Rejeitar (u, v) (formaria ciclo).
4. Parar quando a MST tiver $n - 1$ arestas.

Resultado: A MST sempre terá exatamente $n - 1$ arestas para n vértices.

---

3. Representações de Grafos — Exemplo com o grafo:



3a. Matriz de Adjacência

$$M = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

$M[i][j] = 1$ se existe aresta entre i e j.
Propriedade: a matriz é **simétrica** em grafos não direcionados.

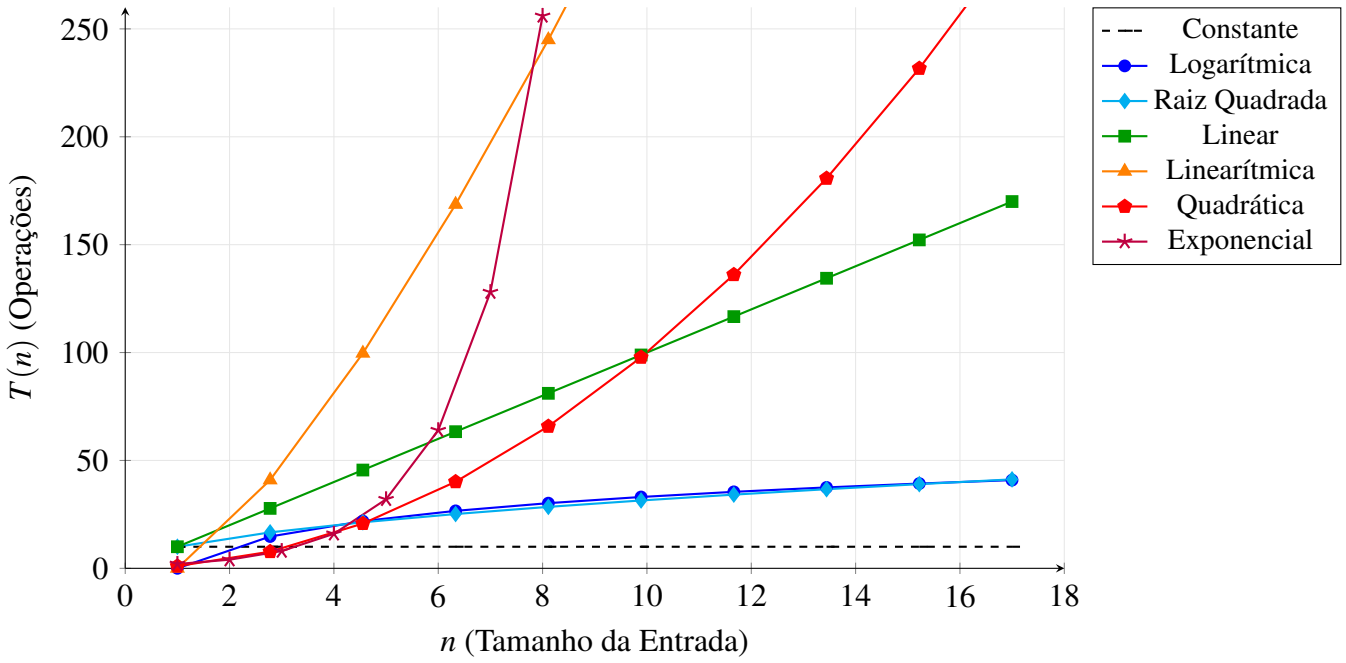
3b. Lista de Adjacência

Vértice	Vizinhos
1	$\rightarrow 2 \rightarrow 3$
2	$\rightarrow 1 \rightarrow 3 \rightarrow 4$
3	$\rightarrow 1 \rightarrow 2 \rightarrow 4$
4	$\rightarrow 2 \rightarrow 3$

Cada vértice armazena a lista dos seus vizinhos diretos.

---

4. Comparativo Visual das Complexidades



5. Tábua de Logaritmos (*Base 10 e Base 2*)

N	$\log_{10}$	$\log_2$	**N**	$\log_{10}$	$\log_2$	**N**	$\log_{10}$	$\log_2$	**N**	$\log_{10}$	$\log_2$	**N**	$\log_{10}$	$\log_2$
10	1.00	3.32	310	2.49	8.28	610	2.79	9.25	910	2.96	9.83	1210	3.08	10.24
20	1.30	4.32	320	2.51	8.32	620	2.79	9.28	920	2.96	9.85	1220	3.09	10.25
30	1.48	4.91	330	2.52	8.37	630	2.80	9.30	930	2.97	9.86	1230	3.09	10.26
40	1.60	5.32	340	2.53	8.41	640	2.81	9.32	940	2.97	9.88	1240	3.09	10.28
50	1.70	5.64	350	2.54	8.45	650	2.81	9.34	950	2.98	9.89	1250	3.10	10.29
60	1.78	5.91	360	2.56	8.49	660	2.82	9.37	960	2.98	9.91	1260	3.10	10.30
70	1.85	6.13	370	2.57	8.53	670	2.83	9.39	970	2.99	9.92	1270	3.10	10.31
80	1.90	6.32	380	2.58	8.57	680	2.83	9.41	980	2.99	9.94	1280	3.11	10.32
90	1.95	6.49	390	2.59	8.61	690	2.84	9.43	990	3.00	9.95	1290	3.11	10.33
100	2.00	6.64	400	2.60	8.64	700	2.85	9.45	1000	3.00	9.97	1300	3.11	10.34
110	2.04	6.78	410	2.61	8.68	710	2.85	9.47	1010	3.00	9.98	1310	3.12	10.36
120	2.08	6.91	420	2.62	8.71	720	2.86	9.49	1020	3.01	9.99	1320	3.12	10.37
130	2.11	7.02	430	2.63	8.75	730	2.86	9.51	1030	3.01	10.01	1330	3.12	10.38
140	2.15	7.13	440	2.64	8.78	740	2.87	9.53	1040	3.02	10.02	1340	3.13	10.39
150	2.18	7.23	450	2.65	8.81	750	2.88	9.55	1050	3.02	10.04	1350	3.13	10.40
160	2.20	7.32	460	2.66	8.85	760	2.88	9.57	1060	3.03	10.05	1360	3.13	10.41
170	2.23	7.41	470	2.67	8.88	770	2.89	9.59	1070	3.03	10.06	1370	3.14	10.42
180	2.26	7.49	480	2.68	8.91	780	2.89	9.61	1080	3.03	10.08	1380	3.14	10.43
190	2.28	7.57	490	2.69	8.94	790	2.90	9.63	1090	3.04	10.09	1390	3.14	10.44
200	2.30	7.64	500	2.70	8.97	800	2.90	9.64	1100	3.04	10.10	1400	3.15	10.45
210	2.32	7.71	510	2.71	8.99	810	2.91	9.66	1110	3.05	10.12	1410	3.15	10.46
220	2.34	7.78	520	2.72	9.02	820	2.91	9.68	1120	3.05	10.13	1420	3.15	10.47
230	2.36	7.85	530	2.72	9.05	830	2.92	9.70	1130	3.05	10.14	1430	3.16	10.48
240	2.38	7.91	540	2.73	9.08	840	2.92	9.71	1140	3.06	10.15	1440	3.16	10.49
250	2.40	7.97	550	2.74	9.10	850	2.93	9.73	1150	3.06	10.17	1450	3.16	10.50
260	2.41	8.02	560	2.75	9.13	860	2.93	9.75	1160	3.06	10.18	1460	3.16	10.51
270	2.43	8.08	570	2.76	9.15	870	2.94	9.76	1170	3.07	10.19	1470	3.17	10.52
280	2.45	8.13	580	2.76	9.18	880	2.94	9.78	1180	3.07	10.20	1480	3.17	10.53
290	2.46	8.18	590	2.77	9.20	890	2.95	9.80	1190	3.08	10.22	1490	3.17	10.54
300	2.48	8.23	600	2.78	9.23	900	2.95	9.81	1200	3.08	10.23	1500	3.18	10.55